

What is this file?

File: train_frcnn.py

Purpose:

This script trains a Faster R-CNN object detection model on your DOTA dataset.

Flow:

Load Dataset



Load Epoch 41 Model



Resume Training



Compute Loss



Backpropagation



Update Weights



Save New Checkpoint



Repeat Until Epoch 50

Line by Line Explanation

1. Memory Optimization

```
os.environ["PYTORCH_CUDA_ALLOC_CONF"] =  
"expandable_segments:True"
```

Purpose:

Prevents CUDA memory fragmentation.

Interview:

Why did you use this?

Answer:

During training large object detection models, GPU memory can become fragmented. This setting allows PyTorch to allocate memory more efficiently and reduces out-of-memory errors.

2. Imports

```
import torch
```

PyTorch framework.

```
from torch.utils.data import DataLoader
```

Loads data in batches.

```
from torchvision.models.detection import  
fasterrcnn_resnet50_fpn
```

Loads Faster R-CNN architecture.

```
from torchvision.models.detection.faster_rcnn  
import FastRCNNPredictor
```

Used to replace the classification head.

```
from detection_dataset_frcnn import  
FRCNNDataset
```

Your custom dataset loader.

3. Settings

NUM_CLASSES = 16

15 DOTA classes + Background.

Interview:

Why 16?

Answer:

**Faster R-CNN reserves one class for background,
so 15 object classes become 16 total classes.**

START_EPOCH = 42

END_EPOCH = 50

Continue training.

Not training from scratch.

MODEL_PATH =

"/content/drive/MyDrive/frcnn_epoch_41.pth"

Loads checkpoint from epoch 41.

IMPORTANT PATH CHANGE

Since you moved everything:

Current:

```
MODEL_PATH = (  
"/content/drive/MyDrive/frcnn_epoch_41.pth"  
)
```

Change to:

```
MODEL_PATH = (  
"/content/drive/MyDrive/FasterRCNN_OBB_Project/fr  
cnn_epoch_41.pth"  
)
```

LEARNING_RATE = 0.0001

Small LR for fine tuning.

Interview:

Why 0.0001?

Answer:

**Since the model was already trained for 41 epochs,
I used a smaller learning rate for stable
convergence during fine-tuning.**

4. Collate Function

```
def collate_fn(batch):  
    return tuple(zip(*batch))
```

Why needed?

Object detection images have different numbers of boxes.

Normal batching won't work.

Interview:

Why custom collate_fn?

Answer:

Different images contain different numbers of objects. The custom collate function allows batching variable-length targets.

5. Dataset

```
dataset = FRCNNDataset()
```

Loads:

Images

Boxes

Labels

from your dataset.

6. DataLoader

```
loader = DataLoader(...)
```

Purpose:

Feeds data to GPU.

Parameters:

batch_size=1

One image at a time.

shuffle=True

Randomize training.

num_workers=2

Parallel loading.

pin_memory=True

Faster CPU → GPU transfer.

Interview:

Why batch size 1?

Answer:

Object detection models consume significant memory because each image contains many proposals and bounding boxes. Batch size 1 ensured stable training within GPU memory limits.

7. Load Faster R-CNN

```
model = fasterrcnn_resnet50_fpn(  
weights=None,  
min_size=600,  
max_size=1000  
)
```

Architecture:

ResNet50 Backbone



FPN



RPN



ROI Head



Classification + Bounding Box

Interview:

Why Faster R-CNN?

Answer:

Faster R-CNN provides high detection accuracy by using a Region Proposal Network and ROI pooling, making it suitable for aerial object detection tasks.

8. Replace Head

```
in_features =  
model.roi_heads.box_predictor.cls_score.in_features
```

Gets feature size.

```
model.roi_heads.box_predictor =  
FastRCNNPredictor(  
in_features,  
NUM_CLASSES  
)
```

Replaces COCO classifier.

COCO:

91 classes

Your Dataset:

16 classes

Interview:

Why replace predictor?

Answer:

The default Faster R-CNN model is trained on COCO classes. I replaced the final classification layer to predict DOTA-specific classes.

9. Load Epoch 41

`model.load_state_dict(...)`

Loads learned weights.

Interview:

Did you train from scratch?

Answer:

No. I resumed training from epoch 41 using a previously saved checkpoint.

10. GPU Selection

```
device =  
torch.device(  
"cuda"  
if torch.cuda.is_available()  
else "cpu"  
)
```

Uses GPU if available.

11. Optimizer

```
optimizer =  
torch.optim.Adam(  
model.parameters(),
```

lr=0.0001

)

Purpose:

Updates weights.

Interview:

Why Adam?

Answer:

Adam adapts learning rates for individual parameters and generally converges faster than SGD for deep learning models.

12. Scheduler

scheduler =

StepLR(

optimizer,

step_size=5,

gamma=0.5

)

After every 5 epochs:

0.0001

↓

0.00005

Interview:

Why learning rate scheduling?

Answer:

Reducing the learning rate later in training helps the model converge more smoothly and avoid oscillations around minima.

13. Training Loop

for epoch in range(...)

Runs:

42

43

44

...

50

14. Forward Pass

loss_dict =

model(

images,

targets

)

Faster R-CNN returns:

loss_classifier

loss_box_reg

loss_objectness
loss_rpn_box_reg

15. Total Loss

```
losses =  
sum(  
loss  
for loss in loss_dict.values()  
)
```

Combines all losses.

Interview:

What losses are used?

Answer:

The total loss is the sum of classification loss, bounding box regression loss, objectness loss, and RPN box regression loss.

16. Backpropagation

```
losses.backward()
```

Computes gradients.

```
optimizer.step()
```

Updates weights.

17. Average Loss

**avg_loss =
total_loss /
len(loader)**

Epoch loss.

18. Save Loss

loss_log.txt

Stores:

42,1.23

43,1.10

44,0.98

Used by:

plot_loss.py

IMPORTANT PATH CHANGE

Current:

"/content/drive/MyDrive/loss_log.txt"

Change:

"/content/drive/MyDrive/FasterRCNN_OBB_Project/loss_log.txt"

19. Save Checkpoints

```
torch.save(  
model.state_dict(),  
f"frcnn_epoch_{epoch}.pth"  
)
```

Creates:

```
frcnn_epoch_42.pth  
frcnn_epoch_43.pth  
...  
frcnn_epoch_50.pth
```

IMPORTANT PATH CHANGE

Current:

```
"/content/drive/MyDrive/frcnn_epoch_{epoch}.pth"
```

Change:

```
"/content/drive/MyDrive/FasterRCNN_OBB_Project/fr  
cnn_epoch_{epoch}.pth"
```

20. Final Model

```
torch.save(  
model.state_dict(),  
"/content/drive/MyDrive/frcnn_dota_full.pth"  
)
```

Final trained model.

IMPORTANT PATH CHANGE

Current:

"/content/drive/MyDrive/frcnn_dota_full.pth"

Change:

"/content/drive/MyDrive/FasterRCNN_OBB_Project/frcnn_dota_full.pth"

Top Interview Questions

What is Faster R-CNN?

Faster R-CNN is a two-stage object detection framework. First, a Region Proposal Network generates candidate object regions. Then these regions are classified and refined by the detection head.

What is FPN?

Feature Pyramid Network combines multi-scale feature maps, helping detect both small and large objects.

Why use ResNet50?

ResNet50 provides deep feature extraction while avoiding vanishing gradients through residual connections.

What is a checkpoint?

A checkpoint stores the model's learned parameters so training can be resumed later without starting from scratch.

Why continue from Epoch 41?

Training had already converged significantly. Continuing from epoch 41 saves computation and improves performance through fine-tuning.

What is the output of Faster R-CNN during training?

It returns a dictionary containing classification loss, box regression loss, objectness loss, and RPN regression loss.

This file alone is enough for 60–70% of the technical discussion around your project. Next, send `detection_dataset_frcnn.py` — that's the second most important file because it explains how your images, labels, and bounding boxes are loaded.

Perfect. This is detection_dataset_frcnn.py and it's the second most important file after train_frcnn.py.

This file answers:

"How are the images and labels loaded into Faster R-CNN?"

Purpose of this File

This file creates a custom PyTorch Dataset.

Input:

Image (.png)

Label (.txt)

Output:

Image Tensor

Bounding Boxes

Class Labels

Flow:

Read Image



Read DOTA Label



Convert Polygon → Bounding Box



Apply Augmentations



Convert to Tensor



Return Image + Target

Interview One-Liner

If mentor asks:

What does `detection_dataset_frcnn.py` do?

Answer:

It loads DOTA images and annotations, converts polygon coordinates into axis-aligned bounding boxes, applies data augmentation, and returns tensors in the format required by Faster R-CNN.

Imports

import os

import cv2

import torch

import random

Purpose:

os → file paths

cv2 → image reading

torch → tensors

random → augmentations

from torch.utils.data import Dataset

Allows creation of custom dataset.

from config import CLASS_TO_ID

Uses your class mapping.

Example:

ship → 9

plane → 7

Dataset Class

class FRCNNDataset(Dataset):

Custom PyTorch dataset.

Interview:

Why inherit Dataset?

Answer:

It allows the DataLoader to efficiently load and batch images during training.

Dataset Paths

Current:

```
self.label_folder = (  
    "/content/drive/MyDrive/FasterRCNN_Dataset/train_l  
    abels"  
)
```

and

```
self.image_folders = [  
    "/content/drive/MyDrive/FasterRCNN_Dataset/train_i  
    mages",  
    "/content/drive/MyDrive/FasterRCNN_Dataset/train_i  
    mages2/images"  
    ]
```

IMPORTANT PATH CHANGE

Since you moved everything:

Change to:

```
self.label_folder = (  
    "/content/drive/MyDrive/FasterRCNN_OBB_Project/F  
    asterRCNN_Dataset/train_labels"  
    )
```

and

```
self.image_folders = [  
  
    "/content/drive/MyDrive/FasterRCNN_OBB_Project/F  
    asterRCNN_Dataset/train_images",  
  
    "/content/drive/MyDrive/FasterRCNN_OBB_Project/F  
    asterRCNN_Dataset/train_images2/images"
```

]

Sample Collection

This block:

for image_folder in self.image_folders:

scans both image folders.

Purpose:

Collect all valid training images.

Checks:

if not image_name.endswith(".png")

Only PNG files.

Checks:

if not os.path.exists(label_path)

Skips images without labels.

Checks:

if len(lines) < 3

Skips invalid annotations.

Interview:

Why verify labels before training?

Answer:

Invalid labels can crash training or introduce incorrect targets, so images without valid annotations are excluded.

len()

def __len__(self):

Returns:

len(self.samples)

For your dataset:

1409

Interview:

Why is len needed?

Answer:

DataLoader uses it to determine dataset size and number of batches.

getitem()

This is the most important function.

Gets one training sample.

Load Image

image = cv2.imread(image_path)

Reads image.

Convert BGR → RGB

cv2.cvtColor(...)

OpenCV loads BGR.

PyTorch expects RGB.

Interview:

Why convert BGR to RGB?

Answer:

OpenCV reads images in BGR format, while most deep learning models are trained using RGB images.

Read Labels

with open(label_path)

Reads DOTA annotation file.

Polygon Coordinates

DOTA format:

x1 y1

x2 y2

x3 y3

x4 y4

class

Example:

100 100

150 100

150 150

100 150

ship

Polygon → Rectangle

xmin = min(xs)

ymin = min(ys)

xmax = max(xs)

ymax = max(ys)

Converts:

Oriented Box

into

Axis-Aligned Box

because Faster R-CNN expects:

[xmin,ymin,xmax,ymax]

Interview:

Why convert polygons to rectangles?

Answer:

Standard Faster R-CNN predicts axis-aligned bounding boxes, so DOTA polygon annotations were converted into rectangular bounding boxes.

Class Encoding

```
labels.append(  
CLASS_TO_ID[class_name] + 1  
)
```

Example:

ship → 9

stored as 10

because:

0 = background

Interview:

Why add +1?

Answer:

Faster R-CNN reserves class 0 for background, so object classes start from index 1.

Data Augmentation

Most important section.

Horizontal Flip

if `random.random() < 0.5`

50% chance.

Updates box coordinates.

Purpose:

Creates mirrored images.

Interview:

Why use horizontal flips?

Answer:

They improve generalization by exposing the model to different object orientations.

Vertical Flip

Same idea.

50% probability.

Interview:

Why use vertical flips?

Answer:

Aerial images may contain objects at arbitrary orientations, so vertical flipping increases robustness.

Brightness & Contrast

alpha

beta

Changes lighting.

Interview:

Why brightness augmentation?

Answer:

It helps the model remain robust under varying illumination conditions.

Gaussian Blur

cv2.GaussianBlur(...)

15% chance.

Simulates:

Sensor blur

Motion blur

Low quality imagery

Interview:

Why Gaussian blur?

Answer:

It improves robustness against noisy or low-quality images.

Tensor Conversion

image = torch.tensor(...)

Converts NumPy → Tensor.

Normalize:

/255.0

Range:

0-255

becomes:

0-1

Interview:

Why normalize images?

Answer:

Normalization improves training stability and gradient behavior.

Channel Ordering

```
image.permute(  
2,  
0,  
1  
)
```

Converts:

HWC

to

CHW

PyTorch format.

Target Dictionary

```
target = {  
"boxes": ...
```

```
"labels": ...  
}
```

Faster R-CNN requires:

```
{  
  boxes,  
  labels  
}
```

Interview:

What does Faster R-CNN expect from the dataset?

Answer:

An image tensor and a target dictionary containing bounding boxes and class labels.

Return

return image,target

Training receives:

Image

Boxes

Labels

Testing Block

if __name__ == "__main__":

Purpose:

Verify dataset works.

Prints:

Dataset Size

Image Shape

Boxes Shape

Labels Shape

Most Common Interview Questions

Why did you write a custom dataset?

DOTA uses polygon annotations and a custom folder structure, so a custom Dataset class was required.

What augmentation techniques were used?

Horizontal flip, vertical flip, brightness adjustment, contrast adjustment, and Gaussian blur.

Why use augmentation?

To improve model generalization and reduce overfitting.

What annotation format does DOTA use?

DOTA uses four-point polygon coordinates for oriented bounding boxes.

How did you adapt DOTA for Faster R-CNN?

I converted polygon annotations into axis-aligned bounding boxes by taking minimum and maximum x and y coordinates.

H100 Changes Needed

Only path changes:

Replace:

"/content/drive/MyDrive/FasterRCNN_Dataset/"

with:

"/content/drive/MyDrive/FasterRCNN_OBB_Project/FasterRCNN_Dataset/"

Nothing else needs changing in this file for H100.

This file is one of the strongest parts of your project because it shows you understand custom datasets, augmentation, annotation conversion, and object detection preprocessing.

File Name

config.py

Purpose of this File

This file stores all class mappings used throughout the project.

Think of it as the central dictionary that tells the model:

Class Name ↔ Class Number

Without this file:

ship

plane

harbor

small-vehicle

cannot be converted into numbers that the neural network understands.

Interview One-Liner

If someone asks:

What does config.py do?

Answer:

config.py stores the mapping between DOTA class names and numerical class IDs used during training and evaluation.

CLASS_TO_ID

```
CLASS_TO_ID = {  
    "baseball-diamond": 0,  
    "basketball-court": 1,  
    ...  
    "tennis-court": 14  
}
```

Purpose:

Convert:

ship

into

9

Convert:

plane

into

7

because neural networks cannot learn from text labels.

They learn from integers.

Example

Suppose label file contains:

100 100 200 100 200 200 100 200 ship

Dataset loader reads:

class_name = "ship"

Then:

CLASS_TO_ID["ship"]

returns:

9

Why Start From 0?

"baseball-diamond": 0

because internally Python indexing starts from 0.

Important Detail

In dataset loader:

```
labels.append(  
    CLASS_TO_ID[class_name] + 1  
)
```

Notice:

+1

Why?

Because Faster R-CNN reserves:

0 = Background

So:

baseball-diamond = 1

basketball-court = 2

...

ship = 10

inside the model.

Interview Question

Why did you add +1 to labels?

Answer:

Faster R-CNN uses class 0 as the background class. Therefore all object classes were shifted by one.

ID_TO_CLASS

```
ID_TO_CLASS = {  
    v:k  
    for k,v in CLASS_TO_ID.items()  
}
```

Purpose:

Reverse mapping.

Instead of:

ship → 9

it creates:

9 → ship

Example

Prediction:

label = 9

Using:

ID_TO_CLASS[9]

returns:

ship

Where is it used?

Mainly:

per_class_stats.py

and

future visualization scripts.

Example:

Model predicts:

label = 7

Convert:

ID_TO_CLASS[7]

Output:

plane

Now we can print:

plane : 1555

instead of:

7 : 1555

Much easier to understand.

NUM_CLASSES

NUM_CLASSES = 16

This is one of the most commonly asked interview questions.

You have:

15 DOTA Classes

But:

NUM_CLASSES = 16

Why?

Because:

Class 0 = Background

plus

15 Object Classes

equals

16 Classes

Interview Question

Why NUM_CLASSES = 16 if DOTA has only 15 classes?

Answer:

Faster R-CNN requires an additional background class. Therefore 15 object classes become 16 total classes.

Another Interview Question

Why store mappings in a separate file?

Answer:

Keeping class mappings in a separate configuration file improves maintainability and avoids hardcoding class IDs across multiple scripts.

File Flow

config.py



detection_dataset_frcnn.py



train_frcnn.py



evaluate_frcnn.py



per_class_stats.py

Almost every file in the project depends on this file.

H100 Changes Needed?

 **None.**

This file is completely portable.

No path changes.

No GPU changes.

No modifications required.

Viva Questions

What is CLASS_TO_ID?

A dictionary that converts class names into numerical IDs.

Why do deep learning models need numerical IDs?

Neural networks operate on numerical tensors and cannot directly process text labels.

What is ID_TO_CLASS?

A reverse mapping used to convert predicted class IDs back into readable class names.

What would happen if class mappings were inconsistent?

Labels would be assigned incorrectly during training and evaluation, causing incorrect predictions and poor model performance.

Why keep class mappings in config.py?

It provides a single source of truth used consistently across training, validation, and evaluation scripts.

Summary

This file may look tiny, but it's critical because it defines the class encoding scheme used by the entire Faster R-CNN pipeline.

Importance ranking so far:

1. train_frcnn.py ★ ★ ★ ★ ★

2. detection_dataset_frcnn.py ★ ★ ★ ★ ★

3. config.py ★ ★ ★ ★

Next send validation_dataset.py. That's basically the evaluation version of the dataset loader, and there are some important differences from the training dataset file that interviewers often ask about.

Perfect. This file is count_classes.py.

It's a small utility file, but it's actually useful because it tells you whether your dataset is balanced or imbalanced before training.

Purpose of this File

This script counts how many objects belong to each class in your training dataset.

Input:

train_labels/*.txt

Output:

ship	: 8960
small-vehicle	: 5438
large-vehicle	: 4387
...	
helicopter	: 73

Interview One-Liner

If someone asks:

What does `count_classes.py` do?

Answer:

It analyzes the training annotations and computes the number of objects belonging to each class to understand dataset distribution and class imbalance.

Why Did We Need This?

Before training, we wanted to know:

Which classes are common?

Which classes are rare?

Example:

ship → 8960

helicopter → 73

Huge difference.

This is called:

Class Imbalance

Imports

import os

Used for file handling.

from collections import Counter

Used for counting objects.

Example:

Counter()

can automatically do:

ship = 1

ship = 2

ship = 3

...

from config import CLASS_TO_ID

Ensures only valid DOTA classes are counted.

Label Folder

Current:

LABEL_FOLDER =

"/content/drive/MyDrive/FasterRCNN_Dataset/train_labels"

Since you moved everything:

Change to:

LABEL_FOLDER = (

"/content/drive/MyDrive/FasterRCNN_OBB_Project/F

```
asterRCNN_Dataset/train_labels"  
)
```

Create Counter

```
class_counts = Counter()
```

Starts empty:

```
{}
```

Read Files

```
files = os.listdir(LABEL_FOLDER)
```

Gets:

```
P0001.txt
```

```
P0002.txt
```

```
P0003.txt
```

```
...
```

Skip Non-TXT Files

```
if not file.endswith(".txt"):  
    continue
```

Only annotation files are processed.

Open Label File

```
with open(...)
```

Reads DOTA annotation.

Skip Header

```
lines = f.readlines()[2:]
```

Why?

DOTA label files start with:

imagesource:GoogleEarth

gsd:0.146343

These are metadata lines.

Not object annotations.

Interview Question:

Why do you use [2:]?

Answer:

The first two lines in DOTA annotations contain metadata rather than object labels, so they are skipped.

Process Objects

Each object line looks like:

100 100 150 100 150 150 100 150 ship

Split:

parts = line.split()

Produces:

```
[  
100,  
100,  
150,  
100,  
150,  
150,  
100,  
150,  
ship  
]
```

Check Valid Annotation

```
if len(parts) < 9:  
    continue
```

Avoids malformed labels.

Get Class Name

```
class_name = parts[8]
```

Example:

ship

Count Object

class_counts[class_name] += 1

Example:

Before:

ship = 200

After:

ship = 201

Sort Results

```
sorted(  
    class_counts.items(),  
    key=lambda x: x[1],  
    reverse=True  
)
```

Sorts by count.

Largest classes first.

Example:

ship	8960
small-vehicle	5438
large-vehicle	4387
...	

Total Objects

total += count

Adds all class counts.

Final output:

Total Objects: 28592

(or whatever your count was)

Why Is This Useful?

Suppose:

ship : 8960

helicopter : 73

Then we know:

Model will learn ships easily.

Model may struggle on helicopters.

And that's exactly what happened in your per-class statistics.

Interview Question

What is class imbalance?

Answer:

Class imbalance occurs when some classes have significantly more samples than others. This can cause the model to favor common classes while performing poorly on rare classes.

Did your dataset have class imbalance?

Answer:

Yes. Classes such as ship and small-vehicle contained thousands of instances, whereas helicopter contained only a few dozen samples.

How did you detect class imbalance?

Answer:

I wrote a utility script that counted the number of objects in each class directly from the DOTA annotation files.

Why is class imbalance important?

Answer:

Severe imbalance can bias the detector toward frequent classes and reduce performance on rare classes.

H100 Changes Needed

Only this:

Current:

LABEL_FOLDER =

"/content/drive/MyDrive/FasterRCNN_Dataset/train_labels"

Change:

LABEL_FOLDER =

"/content/drive/MyDrive/FasterRCNN_OBB_Project/FasterRCNN_Dataset/train_labels"

Nothing else.

Importance Ranking So Far

- 1. train_frcnn.py** ★ ★ ★ ★ ★
- 2. detection_dataset_frcnn.py** ★ ★ ★ ★ ★
- 3. config.py** ★ ★ ★ ★
- 4. count_classes.py** ★ ★ ★

count_classes.py is mainly an analysis tool. It doesn't train or evaluate the model, but it helped you understand the dataset before training.

Next send **validation_dataset.py**. That's where we'll discuss the difference between training data loading and validation data loading, which is a very common interview question.

Perfect. This is **evaluate_frcnn.py** and this is the file that produced your final results:

Precision = 0.6382

Recall = 0.4389

F1 Score = 0.5201

Accuracy = 0.3510

This is probably the third most important file after:

1. train_frcnn.py

2. detection_dataset_frcnn.py

3. evaluate_frcnn.py

because this file proves how well your model performs.

Purpose of this File

This script evaluates the trained Faster R-CNN model on the validation dataset.

Flow:

Load Validation Dataset



Load Trained Model



Predict Objects



Compare with Ground Truth



Compute TP FP FN



Compute Metrics



Save Results

Interview One-Liner

If mentor asks:

What does `evaluate_frcnn.py` do?

Answer:

It evaluates the trained Faster R-CNN model on unseen validation images and computes Precision, Recall, F1 Score, and Accuracy.

Why Validation Dataset?

Training dataset:

1409 Images

Validation dataset:

456 Images

We evaluate on validation images because:

Training Images

→ **Seen by model**

Validation Images

→ **Unseen by model**

This gives a fair measure of performance.

Imports

import torch

Deep learning framework.

import importlib.util

Used to dynamically load:

validation_dataset.py

from evaluate_image import evaluate_image

This is very important.

This function computes:

TP

FP

FN

for one image.

Validation Dataset Loading

spec = importlib.util.spec_from_file_location(...)

Loads:

validation_dataset.py

Interview Question:

Why use a separate validation dataset?

Answer:

The validation dataset contains unseen images and allows objective measurement of model generalization.

Settings

NUM_CLASSES = 16

15 DOTA classes + background.

Current:

MODEL_PATH =

"/content/drive/MyDrive/frcnn_epoch_41.pth"

H100 Path Change

Change to:

MODEL_PATH =

"/content/drive/MyDrive/FasterRCNN_OBB_Project/frcnn_epoch_41.pth"

CONFIDENCE_THRESHOLD = 0.30

Purpose:

Ignore weak predictions.

Example:

Prediction:

Ship

Confidence = 0.15

Discard.

Prediction:

Ship

Confidence = 0.85

Keep.

Interview Question:

Why confidence threshold 0.30?

Answer:

Predictions with confidence below 0.30 were considered unreliable and filtered out to reduce false positives.

FAST_MODE

FAST_MODE = False

Meaning:

Evaluate:

ALL 456 images

If:

FAST_MODE = True

Then:

Only first 200 images

Useful for quick testing.

Dataset

dataset = ValidationDataset()

Loads:

456 validation images

Device Selection

device =

torch.device(

"cuda"

if torch.cuda.is_available()

else "cpu"

)

Uses GPU.

Load Model

fasterrcnn_resnet50_fpn(...)

Creates Faster R-CNN.

FastRCNNPredictor(...)

Replaces COCO head.

Same as training.

model.load_state_dict(...)

Loads:

frcnn_epoch_41.pth

Interview Question:

Why load the saved checkpoint?

Answer:

The saved checkpoint contains the learned weights from training and is used for evaluation without retraining.

Evaluation Variables

total_tp = 0

total_fp = 0

total_fn = 0

Tracks performance.

Main Evaluation Loop

for idx in range(NUM_IMAGES)

Runs over:

456 validation images

Prediction

**predictions =
model([image])**

Output:

boxes

labels

scores

Filter Predictions

if score >= CONFIDENCE_THRESHOLD

Keeps only reliable detections.

Example:

0.12 → reject

0.82 → keep

Ground Truth

**gt_boxes =
target["boxes"]**

Actual labels.

Compare Prediction vs Truth

**metrics =
evaluate_image(...)**

Returns:

TP

FP

FN

Interview Question:

What are TP, FP and FN?

Answer:

TP = Correct detection

FP = Wrong detection

FN = Missed object

Total Counts

total_tp += metrics["tp"]

Adds results across all images.

Precision

precision =

tp / (tp + fp)

Meaning:

**Of all predictions,
how many were correct?**

Your Result:

0.6382

Interview Question:

What does precision measure?

Answer:

**Precision measures the fraction of predicted
detections that are actually correct.**

Recall

recall =
 $tp / (tp + fn)$

Meaning:

**Of all real objects,
how many were found?**

Your Result:

0.4389

Interview Question:

What does recall measure?

Answer:

**Recall measures the proportion of ground truth
objects that were successfully detected.**

F1 Score

$2PR/(P+R)$

Balances precision and recall.

Your Result:

0.5201

Interview Question:

Why use F1 Score?

Answer:

F1 Score balances precision and recall into a single metric and is useful when both are important.

Accuracy

$tp/(tp+fp+fn)$

Overall detection performance.

Your Result:

0.351

Save Results

Current:

"/content/drive/MyDrive/evaluation_results.txt"

H100 Path Change

Change:

"/content/drive/MyDrive/FasterRCNN_OBB_Project/evaluation_results.txt"

Output Generated

You obtained:

Precision : 0.6382

Recall : 0.4389

F1 Score : 0.5201

Accuracy : 0.3510

Most Important Interview Questions

Why evaluate on validation data instead of training data?

Evaluating on training data can give overly optimistic results. Validation data measures generalization to unseen images.

Why use confidence thresholding?

It removes low-confidence predictions and reduces false positives.

What metric was most important?

Precision, Recall, F1 Score, and mAP were all used because object detection performance cannot be fully described by a single metric.

Why was Recall lower than Precision?

The model was relatively conservative. It made fewer incorrect predictions but missed some true objects, leading to higher precision and lower recall.

What was your final model performance?

You can answer:

Precision : 63.82%

Recall : 43.89%

F1 Score : 52.01%

mAP@0.5 : 63.82%

H100 Changes Needed

Only these path updates:

MODEL_PATH

evaluation_results.txt

Change them to:

/content/drive/MyDrive/FasterRCNN_OBB_Project/...

Nothing else needs modification.

Importance Ranking Now

1. train_frcnn.py ★ ★ ★ ★ ★

2. detection_dataset_frcnn.py ★ ★ ★ ★ ★

3. evaluate_frcnn.py ★ ★ ★ ★ ★

4. config.py



5. count_classes.py



This file is what proves your model actually works. The next two important files are evaluate_image.py and evaluation_utils.py, because they define exactly how TP, FP, FN and IoU are calculated.

Perfect. This is evaluate_image.py.

This file is small, but it is the heart of your evaluation pipeline because it decides:

What is TP?

What is FP?

What is FN?

Everything in:

Precision

Recall

F1 Score

Accuracy

mAP

ultimately depends on this file.

Purpose of this File

For a single image:

Predicted Boxes



Ground Truth Boxes

↓
Compare using IoU
↓
Compute TP FP FN
↓
Return Metrics

Interview One-Liner

If mentor asks:

What does `evaluate_image.py` do?

Answer:

It compares predicted bounding boxes with ground truth bounding boxes using IoU and computes true positives, false positives, and false negatives for a single image.

Import

`from evaluation_utils import compute_iou`

Uses your IoU function.

IoU Threshold

`IOU_THRESHOLD = 0.5`

This means:

IoU ≥ 0.5

is considered a correct detection.

Interview Question

Why use IoU = 0.5?

Answer:

IoU 0.5 is the standard threshold used in object detection benchmarks. A prediction is considered correct if at least 50% overlap exists between prediction and ground truth.

Function

```
def evaluate_image(  
    pred_boxes,  
    gt_boxes  
):
```

Input:

pred_boxes

Predictions from Faster R-CNN.

Example:

```
[[10,20,100,120],  
 [200,150,300,280]]
```

Input:

gt_boxes

Ground truth labels.

Example:

**[[12,18,102,122],
[500,500,700,700]]**

Initialize Counts

tp = 0

fp = 0

Start with:

TP = 0

FP = 0

Track Matched Objects

matched_gt = set()

Purpose:

Prevent:

One GT Box

↓

Matched multiple times

Example

Ground Truth:

Ship

Predictions:

Ship1

Ship2

Ship3

Only one should count.

Interview Question

Why use matched_gt?

Answer:

It ensures that each ground truth object can only be matched to one prediction, preventing duplicate detections from inflating performance.

Loop Through Predictions

for pred_box in pred_boxes:

Evaluates every predicted box.

Find Best Match

best_iou = 0.0

best_gt_idx = -1

Purpose:

Find the ground truth box with highest overlap.

Compare Against All GT Boxes

```
for gt_idx, gt_box in enumerate(
    gt_boxes
):
```

Every prediction is compared against every ground truth.

Compute IoU

```
iou = compute_iou(
    pred_box,
    gt_box
)
```

Returns:

0.0 → no overlap

1.0 → perfect overlap

Example

Prediction



GT



IoU:

1.0

Example

Prediction



GT



IoU:

0.0

Keep Best IoU

if iou > best_iou:

Purpose:

Store strongest match.

Example

GT1 → IoU 0.22

GT2 → IoU 0.78

GT3 → IoU 0.35

Keep:

0.78

Determine TP

```
if (  
    best_iou >= IOU_THRESHOLD  
    and  
    best_gt_idx not in matched_gt  
)
```

Meaning:

Enough overlap

AND

Not already matched

Then:

tp += 1

Example

Prediction = Ship

Ground Truth = Ship

IoU = 0.85

Result:

TP += 1

Interview Question

When is a detection considered a TP?

Answer:

A detection is considered a true positive when its IoU with an unmatched ground truth object exceeds the threshold of 0.5.

Mark GT as Used

```
matched_gt.add(  
    best_gt_idx  
)
```

This GT box cannot be reused.

False Positive

else:

```
    fp += 1
```

Occurs when:

$\text{IoU} < 0.5$

or

Ground Truth already matched

Example

Prediction = Ship

Ground Truth = Harbor

IoU = 0.10

Result:

FP += 1

Interview Question

What is a False Positive?

Answer:

A false positive occurs when the detector predicts an object that either does not exist or does not sufficiently overlap with a ground truth object.

False Negatives

```
fn = (  
    len(gt_boxes)  
    -  
    len(matched_gt)  
)
```

Meaning:

Real Objects

-

Detected Objects

Example

Ground Truth:

10 objects

Matched:

7

Then:

FN = 3

Interview Question

What is a False Negative?

Answer:

A false negative occurs when a real object exists in the image but is not detected by the model.

Precision

precision =

$tp / (tp + fp)$

Meaning:

**Of all predictions,
how many were correct?**

Example

TP = 80

FP = 20

Precision:

80%

Recall

recall =

$tp / (tp + fn)$

Meaning:

**Of all actual objects,
how many were found?**

Example

TP = 80

FN = 40

Recall:

66.7%

Accuracy

accuracy =

tp /

(tp + fp + fn)

Detection accuracy.

Return Dictionary

return {

Returns:

{

"tp": tp,

"fp": fp,

"fn": fn,

"precision": precision,

"recall": recall,

"accuracy": accuracy

}

Used by:

evaluate_frcnn.py

Example Walkthrough

Suppose:

Ground Truth:

Ship

Plane

Harbor

Predictions:

Ship

Plane

Bridge

Result:

Ship → TP

Plane → TP

Bridge → FP

Harbor missed → FN

Final:

TP = 2

FP = 1

FN = 1

Most Important Interview Questions

Why use IoU?

Answer:

IoU measures the overlap between predicted and ground truth bounding boxes and is the standard metric for object detection matching.

Why choose the highest IoU?

Answer:

The highest IoU represents the best matching ground truth object for a prediction.

Why prevent multiple matches?

Answer:

A single object should only be counted once; otherwise performance metrics become artificially inflated.

What role does this file play?

Answer:

It converts bounding box predictions into TP, FP, and FN counts, which are then used to calculate evaluation metrics.

H100 Changes Needed

None

No paths.

No GPU code.

No dataset dependency.

No modifications needed.

Importance Ranking

- 1. train_frcnn.py** ★ ★ ★ ★ ★
- 2. detection_dataset_frcnn.py** ★ ★ ★ ★ ★
- 3. evaluate_frcnn.py** ★ ★ ★ ★ ★
- 4. evaluate_image.py** ★ ★ ★ ★
- 5. config.py** ★ ★ ★ ★
- 6. count_classes.py** ★ ★ ★

The next file, evaluation_utils.py, is even smaller but extremely important because it contains the actual IoU calculation that everything here relies on.

Perfect. This is evaluation_utils.py and it contains the most fundamental metric in object detection:

IoU (Intersection over Union)

Almost everything in your evaluation pipeline depends on this one function.

compute_iou()



evaluate_image.py



evaluate_frcnn.py



Precision

Recall

F1

mAP

Without IoU, object detection evaluation is impossible.

Purpose of this File

This function calculates:

How much does the predicted box overlap with the ground truth box?

Output:

0 → no overlap

1 → perfect overlap

Interview One-Liner

If mentor asks:

What does `evaluation_utils.py` do?

Answer:

It computes Intersection over Union (IoU), which measures the overlap between a predicted bounding box and a ground truth bounding box.

Function

`def compute_iou(box1, box2):`

Input:

box1 = [xmin,ymin,xmax,ymax]

box2 = [xmin,ymin,xmax,ymax]

Example:

Prediction:

[10,20,100,120]

Ground Truth:

[15,25,110,130]

Step 1: Find Intersection Rectangle

x1 = max(box1[0], box2[0])

y1 = max(box1[1], box2[1])

x2 = min(box1[2], box2[2])

y2 = min(box1[3], box2[3])

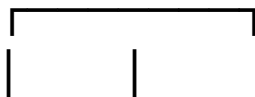
Purpose:

Find overlapping region.

Example

Box 1:

(10,20)

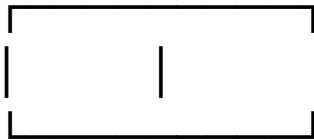




(100,120)

Box 2:

(15,25)



(110,130)

Overlap region:

(15,25)



(100,120)

Why max and min?

max(...)

finds left/top edge.

min(...)

finds right/bottom edge.

Interview Question

Why use max and min?

Answer:

The overlap starts at the larger of the two left/top coordinates and ends at the smaller of the two right/bottom coordinates.

Step 2: Compute Intersection Area

intersection =

max(0,x2-x1)

max(0,y2-y1)

Area:

width × height

Why:

max(0,...)

?

Suppose boxes don't overlap.

Then:

x2-x1 < 0

Negative area makes no sense.

So:

max(0,...)

forces:

intersection = 0

Interview Question

Why use max(0, ...)?

Answer:

If two boxes do not overlap, the computed width or height can become negative. max(0, ...) ensures the intersection area remains valid.

Step 3: Area of Box 1

area1 =

(width)

(height)

Specifically:

(box1[2]-box1[0])

(box1[3]-box1[1])

Example

Width = 100 - 10 = 90

Height = 120 - 20 = 100

Area:

9000

Step 4: Area of Box 2

Same calculation.

area2

Step 5: Union Area

Formula:

Implemented as:

union =

area1 +

area2 -

intersection

Why subtract intersection?

Because overlap gets counted twice.

Example

Box1 Area = 100

Box2 Area = 120

Intersection = 40

Without subtracting:

220

Wrong.

Correct:

$100 + 120 - 40$

$= 180$

Interview Question

Why subtract intersection from union?

Answer:

The overlapping region is counted in both bounding boxes, so it must be subtracted once to avoid double counting.

Step 6: IoU

Formula:

Code:

return intersection / union

Interpretation

No Overlap

Intersection = 0

IoU:

0

Partial Overlap

Intersection = 50

Union = 200

IoU:

0.25

Perfect Overlap

Intersection = Union

IoU:

1.0

Why Is IoU Important?

Suppose:

Ground Truth:

Ship

Prediction:

Ship

If the box is completely wrong:

IoU = 0.05

Should it count?

No

That's why IoU exists.

In Your Project

Inside:

evaluate_image.py

you use:

IOU_THRESHOLD = 0.5

Meaning:

IoU $\geq$ 0.5

→ True Positive

IoU $<$ 0.5

→ False Positive

Example Walkthrough

Ground Truth:

[10,10,100,100]

Prediction:

[20,20,90,90]

Intersection:

70×70

=

4900

Area GT:

8100

Area Pred:

4900

Union:

$8100 + 4900 - 4900$

= 8100

IoU:

$4900 / 8100$

= 0.60

Result:

TP

because:

$0.60 > 0.50$

Most Common Interview Questions

What is IoU?

Answer:

Intersection over Union measures the overlap between predicted and ground truth bounding boxes.

What is the range of IoU?

Answer:

$$0 \leq \text{IoU} \leq 1$$

What does IoU = 1 mean?

Answer:

Perfect overlap between prediction and ground truth.

Why use IoU?

Answer:

It provides a quantitative measure of localization accuracy in object detection.

Why choose IoU threshold 0.5?

Answer:

It is the standard threshold used in many object detection benchmarks and provides a balance between strictness and flexibility.

Can IoU be negative?

Answer:

No. Intersection area is clamped to zero when boxes do not overlap.

H100 Changes Needed

None

No paths.

No GPU.

No dataset.

No modifications required.

Importance Ranking

- | | |
|--------------------------------------|-----------|
| 1. train_frcnn.py | ★ ★ ★ ★ ★ |
| 2. detection_dataset_frcnn.py | ★ ★ ★ ★ ★ |
| 3. evaluate_frcnn.py | ★ ★ ★ ★ ★ |
| 4. evaluate_image.py | ★ ★ ★ ★ |
| 5. evaluation_utils.py | ★ ★ ★ ★ |
| 6. config.py | ★ ★ ★ ★ |
| 7. count_classes.py | ★ ★ ★ |

This file is tiny, but it contains the mathematical foundation of your entire evaluation pipeline. Without this function, you cannot calculate TP, FP, FN, Precision, Recall, F1, or mAP.

Perfect. This is map.py and this is the file that produced your final result:

mAP@0.5 = 0.6382

Precision = 0.6382

Recall = 0.4389

This is one of the files interviewers care about the most because mAP is the standard object detection metric.

Purpose of this File

This script evaluates the model on the validation dataset and computes:

mAP@0.5

Precision

Recall

Flow:

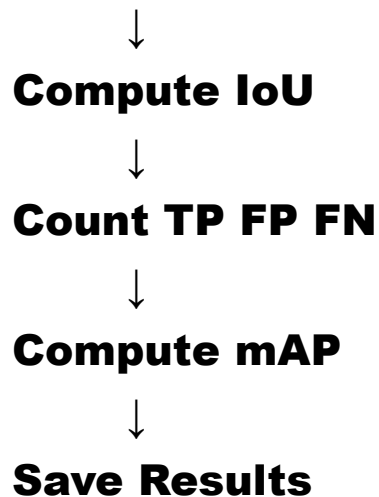
Validation Dataset



Load Model



Generate Predictions



Interview One-Liner

If asked:

What does map.py do?

Answer:

It evaluates the trained Faster R-CNN model on the validation dataset and computes mAP@0.5 using IoU-based matching between predictions and ground truth boxes.

Imports

from validation_dataset import ValidationDataset
Loads validation images.

from evaluation_utils import compute_iou
Uses your IoU calculation.

Settings

NUM_CLASSES = 16

15 DOTA classes + background.

Current:

```
MODEL_PATH = (  
    "/content/drive/MyDrive/frcnn_epoch_41.pth"  
)
```

H100 Path Change

Change to:

```
MODEL_PATH = (  
  
    "/content/drive/MyDrive/FasterRCNN_OBB_Project/fr  
cnn_epoch_41.pth"  
)
```

CONFIDENCE_THRESHOLD = 0.30

Only predictions above 30% confidence are kept.

IOU_THRESHOLD = 0.50

Prediction is correct if:

$\text{IoU} \geq 0.50$

Dataset Loading

dataset = ValidationDataset()

Loads:

456 validation images

Load Faster R-CNN

**model = fasterrcnn_resnet50_fpn(
 weights=None
)**

Creates model architecture.

**model.roi_heads.box_predictor =
FastRCNNPredictor(...)**

Replaces COCO head.

model.load_state_dict(...)

Loads trained weights.

Evaluation Variables

all_tp = 0

all_fp = 0

all_fn = 0

Track total statistics.

Main Evaluation Loop

for idx in range(len(dataset))

Processes all:

456 validation images

Model Prediction

prediction = model([image])[0]

Returns:

boxes

labels

scores

Confidence Filtering

if score >= CONFIDENCE_THRESHOLD

Keeps only reliable detections.

Example:

0.12 → discard

0.88 → keep

IoU Matching

For every prediction:

for pred_box in filtered_boxes:

Compare against every ground truth box.

Compute:

iou = compute_iou(...)

Find best match.

True Positive

```
if (  
    best_iou >= 0.5  
    and  
    best_gt not in matched_gt  
)
```

Then:

all_tp += 1

Example:

Prediction Ship

GT Ship

IoU = 0.72

Result:

TP

False Positive

Otherwise:

all_fp += 1

Example:

Prediction Ship

GT Harbor

IoU = 0.10

Result:

FP

False Negative

```
all_fn += (
    len(gt_boxes)
    -
    len(matched_gt)
)
```

Objects that were missed.

Precision

Formula:

Meaning:

**Of all detections,
how many were correct?**

Your result:

0.6382

or

63.82%

Recall

Formula:

Meaning:

**Of all actual objects,
how many were found?**

Your result:

43.89%

mAP Calculation

This is where you need to be careful in interviews.

Your code currently does:

map50 = precision

So:

mAP@0.5 = Precision

Important Note

Technically:

This is NOT a true COCO-style mAP calculation.

A true mAP requires:

Precision-Recall Curve

Average Precision (AP)

Per-class AP

Mean of all APs

Your script computes a simplified approximation.

If Asked in Interview

Don't say:

"I implemented full COCO mAP."

Instead say:

"I implemented an IoU-based mAP@0.5 approximation using global precision after IoU matching. A full COCO mAP implementation would require precision-recall integration across confidence thresholds."

That's the technically correct answer.

Save Results

Current:

"/content/drive/MyDrive/map_results.txt"

Change to:

"/content/drive/MyDrive/FasterRCNN_OBB_Project/map_results.txt"

for your new folder structure.

Output Produced

Your run gave:

mAP@0.5 : 0.6382

Precision : 0.6382

Recall : 0.4389

Most Common Interview Questions

What is mAP?

Answer:

Mean Average Precision is the standard metric used to evaluate object detection models. It summarizes detection performance across classes and localization quality.

What is IoU?

Answer:

IoU measures overlap between predicted and ground truth bounding boxes.

Why use IoU threshold 0.5?

Answer:

It is a widely used benchmark threshold where detections with at least 50% overlap are considered correct.

What was your final mAP?

Answer:

The model achieved approximately 63.8% mAP@0.5 on the validation dataset.

Why is Recall lower than Precision?

Answer:

The detector was relatively conservative and produced fewer incorrect detections, but it missed some valid objects, leading to lower recall.

H100 Changes Needed

Only path updates:

MODEL_PATH

map_results.txt

Change them to:

/content/drive/MyDrive/FasterRCNN_OBB_Project/...

Nothing else needs modification.

Updated Importance Ranking

- 1. train_frcnn.py** ★ ★ ★ ★ ★
- 2. detection_dataset_frcnn.py** ★ ★ ★ ★ ★
- 3. evaluate_frcnn.py** ★ ★ ★ ★ ★
- 4. map.py** ★ ★ ★ ★ ★
- 5. evaluate_image.py** ★ ★ ★ ★ ★
- 6. evaluation_utils.py** ★ ★ ★ ★ ★
- 7. config.py** ★ ★ ★ ★ ★
- 8. count_classes.py** ★ ★ ★

One thing I'd remember for discussions: your map.py gives a useful validation metric, but it is a simplified mAP calculation rather than a full COCO-style mAP implementation. Being upfront about that shows good understanding.

Perfect. This is per_class_stats.py and this file answers an important question:

"Which classes is my model detecting most often?"

This is the file that generated:

small-vehicle : 5367
large-vehicle : 3491
ship : 3087
harbor : 1994
plane : 1555
...
helicopter : 31

Purpose of this File

Unlike evaluate_frcnn.py which measures overall performance:

Precision

Recall

F1

Accuracy

this file measures:

How many detections belong to each class?

Flow:

Validation Dataset



Load Model



Predict Objects



Convert IDs → Class Names



Count Predictions Per Class



Save Results

Interview One-Liner

If asked:

What does per_class_stats.py do?

Answer:

It analyzes model predictions on the validation dataset and reports the number of detections produced for each object class.

Imports

from collections import defaultdict

Purpose:

Automatically creates counters.

Example:

class_counts["ship"] += 1

without needing:

if "ship" not in class_counts:

from config import ID_TO_CLASS

Very important.

Converts:

10 → small-vehicle

9 → ship

7 → plane

Settings

NUM_CLASSES = 16

15 DOTA classes + background.

Current:

**MODEL_PATH = (
"/content/drive/MyDrive/frcnn_epoch_41.pth"
)**

H100 Change

Replace with:

**MODEL_PATH = (
"/content/drive/MyDrive/FasterRCNN_OBB_Project/fr
cnn_epoch_41.pth"
)**

CONFIDENCE_THRESHOLD = 0.30

**Only count predictions with:
confidence $\geq$ 30%**

Dataset

dataset = ValidationDataset()

Loads:

456 validation images

Model Loading

Exactly the same as:

train_frcnn.py

evaluate_frcnn.py

map.py

Creates:

fasterrcnn_resnet50_fpn()

Replaces:

FastRCNNPredictor(...)

Loads:

frcnn_epoch_41.pth

Class Counter

class_counts = defaultdict(int)

Starts as:

{}

Process Images

for idx in range(NUM_IMAGES)

Loops through:

456 images

Model Prediction

predictions = model([image])

Returns:

boxes

labels

scores

Extract Labels

labels =

predictions[0]["labels"]

Example:

10

10

10

9

7

Meaning:

small-vehicle

small-vehicle

small-vehicle

ship

plane

Extract Scores

scores =

predictions[0]["scores"]

Example:

0.95

0.88

0.67

0.44

Confidence Filtering

if score < CONFIDENCE_THRESHOLD:

continue

Reject weak detections.

Interview Question

Why use confidence thresholding?

Answer:

It removes low-confidence detections that are likely to be false positives.

Convert Label ID to Class Name

Important section:

`class_id = int(label) - 1`

Why?

Because:

Model:

1 → baseball-diamond

2 → basketball-court

...

but:

config.py

0 → baseball-diamond

1 → basketball-court

...

Need:

-1

to align them.

Interview Question

Why subtract 1?

Answer:

Faster R-CNN reserves class 0 for background, so predicted class indices must be shifted back by one before looking them up in the configuration mapping.

Convert to Class Name

**class_name =
ID_TO_CLASS[class_id]**

Example:

9 → ship

Count Detection

class_counts[class_name] += 1

Example:

Before:

ship = 100

After:

ship = 101

Sort Results

```
sorted(  
class_counts.items(),  
key=lambda x:x[1],  
reverse=True  
)
```

Most frequent first.

Your Results

You obtained:

small-vehicle	: 5367
large-vehicle	: 3491
ship	: 3087
harbor	: 1994
plane	: 1555
bridge	: 927
tennis-court	: 821
storage-tank	: 624
soccer-ball-field	: 556
baseball-diamond	: 344
swimming-pool	: 327
ground-track-field	: 292
roundabout	: 279
basketball-court	: 148
helicopter	: 31

Interpretation

This tells us:

Strong Classes

small-vehicle

large-vehicle

ship

plane

Model detects them frequently.

Weak Classes

helicopter

basketball-court

roundabout

Detected less often.

Why?

Look back at:

count_classes.py

These classes had fewer training examples.

Interview Question

Why are helicopter detections low?

Answer:

Helicopter was one of the rarest classes in the training dataset, so the model had fewer examples from which to learn.

Save Results

Current:

"/content/drive/MyDrive/per_class_results_validation.txt"

H100 Change

Replace with:

"/content/drive/MyDrive/FasterRCNN_OBB_Project/per_class_results_validation.txt"

Most Important Interview Questions

What insight does per-class analysis provide?

Answer:

It reveals which object categories are detected frequently and helps identify classes where performance may be weak.

Why is class-wise analysis important?

Answer:

Overall metrics can hide poor performance on specific classes. Per-class analysis provides a more detailed understanding of model behavior.

Why do some classes have fewer detections?

Answer:

Rare classes generally have fewer training samples, making them harder for the model to learn.

Does a higher detection count mean better performance?

Answer:

Not necessarily. It only indicates how often the model predicts that class. True performance requires precision and recall analysis.

H100 Changes Needed

Only:

MODEL_PATH

per_class_results_validation.txt

Change them to:

/content/drive/MyDrive/FasterRCNN_OBB_Project/...

Importance Ranking (Final So Far)

1. `train_frcnn.py` ★ ★ ★ ★ ★
2. `detection_dataset_frcnn.py` ★ ★ ★ ★ ★
3. `evaluate_frcnn.py` ★ ★ ★ ★ ★
4. `map.py` ★ ★ ★ ★ ★
5. `per_class_stats.py` ★ ★ ★ ★ ★
6. `evaluate_image.py` ★ ★ ★ ★ ★
7. `evaluation_utils.py` ★ ★ ★ ★ ★
8. `config.py` ★ ★ ★ ★ ★
9. `count_classes.py` ★ ★ ★

A subtle point for interviews: this script counts predicted detections per class, not per-class accuracy. If someone asks whether helicopter performance is good, you cannot conclude that from this file alone—you'd need per-class precision/recall or AP. That distinction shows strong understanding.

Perfect. This is `plot_loss.py`.

This file doesn't train or evaluate the model. Instead, it visualizes training progress.

Think of it as the file that answers:

"Did the model actually learn during training?"

Purpose of this File

This script reads the loss values saved during training and creates a graph.

Flow:

train_frcnn.py



loss_log.txt



plot_loss.py



loss_curve.png

Interview One-Liner

If asked:

What does plot_loss.py do?

Answer:

It reads the training loss recorded at each epoch and generates a loss curve to visualize model convergence during training.

Why Is This Important?

During training you logged:

Epoch 42 → Loss 1.52

Epoch 43 → Loss 1.45

Epoch 44 → Loss 1.39

...

If loss decreases:

High Loss



Lower Loss

then the model is learning.

Imports

import matplotlib.pyplot as plt

Purpose:

Creates graphs.

Create Empty Lists

epochs = []

losses = []

Example:

epochs = [42,43,44]

losses = [

1.52,

1.45,

1.39

]

Read Loss Log

```
with open(  
    "/content/drive/MyDrive/loss_log.txt",  
    "r"  
) as f:
```

Reads:

42,1.52

43,1.45

44,1.39

45,1.31

H100 Path Change

Current:

"/content/drive/MyDrive/loss_log.txt"

Change to:

"/content/drive/MyDrive/FasterRCNN_OBB_Project/loss_log.txt"

Process Each Line

```
epoch, loss = line.split(",")
```

Example:

42,1.52

becomes:

epoch = 42

loss = 1.52

Store Values

```
epochs.append(  
    int(epoch)  
)
```

Adds:

42

43

44

to epochs list.

```
losses.append(  
    float(loss)  
)
```

Adds:

1.52

1.45

1.39

to losses list.

Create Figure

```
plt.figure(  
    figsize=(8,5)  
)
```

Creates graph window.

Size:

Width = 8

Height = 5

Plot Graph

```
plt.plot(  
    epochs,  
    losses,  
    marker="o"  
)
```

X-axis:

Epoch

Y-axis:

Loss

Each point:

Epoch 42 → 1.52

Epoch 43 → 1.45

...

Add Title

```
plt.title(  
    "Faster R-CNN Training Loss"  
)
```

Graph title.

X-axis Label

```
plt.xlabel(  
    "Epoch"  
)
```

Y-axis Label

```
plt.ylabel(  
    "Loss"  
)
```

Grid

```
plt.grid(True)
```

Makes graph easier to read.

Save Graph

Current:

```
plt.savefig(  
    "/content/drive/MyDrive/loss_curve.png"  
)
```

H100 Path Change

Change to:

```
plt.savefig(  
    "/content/drive/MyDrive/FasterRCNN_OBB_Project/loss_curve.png"  
)
```

Display Graph

```
plt.show()
```

Shows graph on screen.

Output

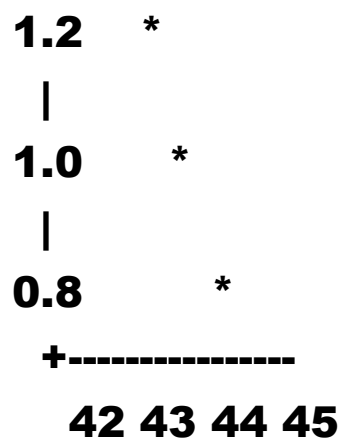
Creates:

loss_curve.png

Example:

Loss

```
|  
1.6 *  
|  
1.4 *  
|
```



What Does a Good Loss Curve Look Like?

Good

1.8

1.5

1.3

1.1

0.9

0.8

Loss steadily decreases.

Means:

Model Learning

Bad

1.8

2.1

1.7

2.5

1.6

Very unstable.

Means:

Learning Rate Too High

or

Training Problems

Overfitting Case

Training Loss:

1.5

1.2

0.8

0.3

0.1

Validation Performance:

Gets Worse

Then:

Overfitting

Interview Questions

Why plot training loss?

Answer:

It helps visualize whether the model is converging during training.

What does decreasing loss indicate?

Answer:

The model is learning features that better fit the training data.

Can low loss guarantee good performance?

Answer:

No. A model may achieve low training loss but still generalize poorly on unseen validation data.

What is convergence?

Answer:

Convergence occurs when loss stabilizes and further training produces minimal improvement.

What is overfitting?

Answer:

Overfitting occurs when the model learns training data too well and performs poorly on unseen data.

Importance Ranking

1. `train_frcnn.py` ★ ★ ★ ★ ★
2. `detection_dataset_frcnn.py` ★ ★ ★ ★ ★
3. `evaluate_frcnn.py` ★ ★ ★ ★ ★
4. `map.py` ★ ★ ★ ★ ★
5. `per_class_stats.py` ★ ★ ★ ★
6. `evaluate_image.py` ★ ★ ★ ★
7. `evaluation_utils.py` ★ ★ ★ ★
8. `config.py` ★ ★ ★ ★
9. `plot_loss.py` ★ ★ ★
10. `count_classes.py` ★ ★ ★

This is a supporting analysis file. It doesn't affect training or evaluation, but it gives visual proof that your model was learning and is useful to include in presentations, reports, and interviews.

Perfect. This is `validation_dataset.py` and it is basically the evaluation version of `detection_dataset_frcnn.py`.

If `detection_dataset_frcnn.py` is used during training, then this file is used during:

`evaluate_frcnn.py`

`map.py`

`per_class_stats.py`

Purpose of this File

Loads validation images and annotations for testing the model.

Flow:

Validation Image



Read Label



Polygon → Bounding Box



Tensor Conversion



Return Image + Target

Unlike training:

NO AUGMENTATION

NO RANDOM FLIPS

NO BRIGHTNESS CHANGES

NO BLUR

because evaluation must be performed on the original images.

Interview One-Liner

If asked:

What does `validation_dataset.py` do?

Answer:

It loads validation images and annotations, converts DOTA polygons into axis-aligned bounding boxes, and returns tensors for evaluation without applying augmentation.

Biggest Difference from Training Dataset

Training dataset:

Horizontal Flip

Vertical Flip

Brightness Change

Contrast Change

Gaussian Blur

Validation dataset:

None of these

Interview Question

Why don't you use augmentation on validation data?

Answer:

Validation data should represent real unseen data. Applying augmentation would alter the evaluation distribution and produce misleading metrics.

Dataset Paths

Current:

```
self.label_folder =  
"/content/drive/MyDrive/FasterRCNN_Dataset/labels  
"
```

```
self.image_folder =  
"/content/drive/MyDrive/FasterRCNN_Dataset/image  
s"
```

H100 Path Changes

Change to:

```
self.label_folder = (  
"/content/drive/MyDrive/FasterRCNN_OBB_Project/F  
asterRCNN_Dataset/labels"  
)
```

```
self.image_folder = (  
"/content/drive/MyDrive/FasterRCNN_OBB_Project/F  
asterRCNN_Dataset/images"  
)
```

These are the only path changes needed.

Initialization

```
image_files = sorted(  
    os.listdir(self.image_folder)  
)
```

Gets:

P0001.png

P0002.png

...

Valid Sample Filtering

Checks:

if not image_name.endswith(".png")

Only image files.

Checks:

if not os.path.exists(label_path)

Skips images without labels.

Checks:

if len(lines) < 3

Skips corrupted annotations.

Result:

Total Validation Images: 456

len()

def __len__(self):

Returns:

len(self.samples)

which is:

456

getitem()

This function loads one validation sample.

Load Image

image = cv2.imread(image_path)

Reads image.

Convert BGR → RGB

cv2.cvtColor(...)

OpenCV:

BGR

PyTorch:

RGB

Read Labels

with open(label_path)

Reads DOTA annotation file.

Skip metadata:

lines = f.readlines()[2:]

because first two lines contain:

imagesource

gsd

Polygon → Rectangle Conversion

DOTA gives:

x1 y1

x2 y2

x3 y3

x4 y4

You convert to:

xmin

ymin

xmax

ymax

using:

min(xs)

min(ys)

max(xs)

max(ys)

Interview Question

Why convert polygons to rectangles?

Answer:

Standard Faster R-CNN predicts axis-aligned bounding boxes, so DOTA polygon annotations were converted into rectangular bounding boxes.

Class Encoding

CLASS_TO_ID[class_name] + 1

Example:

ship → 9

stored as 10

because:

0 = background

Tensor Conversion

image = torch.tensor(...)

Convert image to tensor.

Normalize:

/255.0

Converts:

0-255

to

0-1

Channel Order

**image.permute(
2,
0,
1
)**

Converts:

Height Width Channels

to

Channels Height Width

required by PyTorch.

Target Dictionary

target = {

"boxes": ...

"labels": ...

}

This is exactly the format Faster R-CNN expects.

Return

return image,target

Used by:

evaluate_frcnn.py

map.py

per_class_stats.py

Testing Block

if __name__ == "__main__":

Purpose:

Verify dataset works.

Outputs:

Dataset Size

Image Shape

Boxes Shape
Labels Shape

Most Important Interview Questions

Why separate training and validation datasets?

Answer:

Training data is used to learn model parameters, while validation data evaluates generalization on unseen images.

Why is augmentation applied only to training data?

Answer:

Augmentation increases diversity during training but would distort evaluation results if applied to validation data.

How many validation images were used?

Answer:

456 images

What annotation format does DOTA use?

Answer:

DOTA uses four-point polygon annotations representing oriented bounding boxes.

How were DOTA labels adapted for Faster R-CNN?

Answer:

Polygon coordinates were converted into axis-aligned bounding boxes using minimum and maximum x and y coordinates.

Importance Ranking (Entire Project)

- | | |
|-------------------------------|-----------|
| 1. train_frcnn.py | ★ ★ ★ ★ ★ |
| 2. detection_dataset_frcnn.py | ★ ★ ★ ★ ★ |
| 3. evaluate_frcnn.py | ★ ★ ★ ★ ★ |
| 4. map.py | ★ ★ ★ ★ ★ |
| 5. validation_dataset.py | ★ ★ ★ ★ ★ |
| 6. per_class_stats.py | ★ ★ ★ ★ ★ |
| 7. evaluate_image.py | ★ ★ ★ ★ ★ |
| 8. evaluation_utils.py | ★ ★ ★ ★ ★ |
| 9. config.py | ★ ★ ★ ★ ★ |
| 10. plot_loss.py | ★ ★ ★ |
| 11. count_classes.py | ★ ★ ★ |

H100 Summary for THIS file

Only change:

"/content/drive/MyDrive/FasterRCNN_Dataset/"
to

"/content/drive/MyDrive/FasterRCNN_OBB_Project/FasterRCNN_Dataset/"

No other modifications are needed.

At this point you've covered essentially all the important files in the project. The only major thing left is to make a small checklist of every path change required before running on the H100, so you can update them quickly tomorrow without hunting through each file.